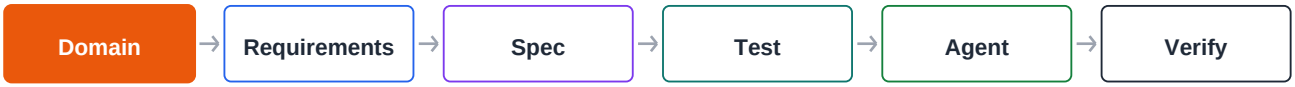
Lezione 2

Domain Discovery: capire il problema prima di specificarlo

Percorso intensivo di AI Software Engineering



Durata indicativa: 30-45 minuti. Obiettivo: imparare a ridurre l'incertezza del dominio prima che venga trasformata in requisiti, specifiche e codice.

Questa lezione continua il principio della Lezione 1: l'AI è un implementatore potente, ma non è l'autorità sul dominio. Prima di chiedere a un agente di costruire, dobbiamo capire che cosa stiamo davvero costruendo e perché.

1. Che cos'è davvero il Domain Discovery

Domain Discovery è il lavoro con cui il team costruisce una comprensione condivisa di un dominio prima di congelarla in requisiti o specifiche. Non è una fase burocratica e non coincide con il disegno di classi, tabelle o API. È un processo di apprendimento collaborativo.

- Osservare: ricostruire come il lavoro accade davvero, non come “dovrebbe” accadere nel documento di processo.
- Dare nomi: trovare termini precisi e condivisi per gli oggetti, gli eventi, le decisioni e le eccezioni del dominio.
- Cercare attrito: parole sovraccariche, “dipende”, eccezioni, workaround, regole implicite e responsabilità poco chiare.
- Disegnare confini: capire dove un concetto smette di avere lo stesso significato e dove potrebbe emergere un Bounded Context.
- Rendere esplicita l'incertezza: distinguere KNOWN, ASSUMED, UNKNOWN e DECIDED invece di mascherare le lacune con una pseudo-specifica.

> *Il risultato del Discovery non è una spec definitiva. È un modello del problema abbastanza nitido da consentire il passaggio alla Requirements Engineering con meno ambiguità.*

2. Discovery, Requirements e Solution Design non sono la stessa cosa

Fase	Domanda principale	Output tipico
Domain Discovery	Che cosa succede nel mondo del problema?	linguaggio, eventi, attori, regole, confini, domande
Requirements	Che cosa deve essere vero o osservabile nel prodotto?	requisiti, acceptance criteria, NFR, priorità
Specification	Quali decisioni rendono il comportamento sufficientemente preciso?	spec, contratti, seam di test, out-of-scope
Solution Design	Come lo implementiamo in questa architettura?	moduli, API, dati, adapter, componenti

Errore comune: passare direttamente da “ho un'idea” a “dammi lo schema SQL”. In quel momento una decisione tecnica diventa implicitamente una decisione di dominio. Il database inizia a dettare il vocabolario anziché rappresentarlo.

3. Le lenti DDD utili nel Discovery

In questa lezione usiamo soprattutto il DDD strategico. Entity, Value Object e Aggregate diventeranno importanti più avanti, ma durante il Discovery ci servono prima di tutto quattro concetti.

Concetto	Uso nel Discovery
Dominio	l'area di conoscenza e attività che il software deve supportare
Ubiquitous Language	il linguaggio rigoroso condiviso da esperti e sviluppatori e usato anche nel software
Subdomain	una parte del problema con obiettivi, regole e competenze proprie
Bounded Context	il confine entro cui un modello e il suo linguaggio sono coerenti

Martin Fowler riassume l'Ubiquitous Language come una lingua comune e rigorosa tra sviluppatori e utenti; inoltre sottolinea che il linguaggio deve evolvere mentre cresce la comprensione del dominio. Per i Bounded Context, il punto chiave è che un grande dominio spesso non può avere un unico modello coerente: termini uguali possono avere modelli differenti in contesti differenti.

Regola pratica: se due persone usano la stessa parola con significati diversi, non hai solo un problema di naming. Potresti avere un problema di modello o di confine.

4. Una sequenza di Discovery che funziona bene

- Parti dai casi reali. Chiedi “raccontami l'ultima volta che è successo”.
- Segui gli eventi. Cosa è successo prima? Cosa ha fatto scattare la decisione? Cosa è successo dopo?
- Fai emergere le regole. “Quando succede X, chi decide Y e in base a cosa?”
- Stressa il caso felice. Cerca rifiuti, timeout, correzioni, rollback, eccezioni e autorizzazioni.
- Segna le parole pericolose. “Cliente”, “ordine”, “ticket”, “utente”, “stato” sono spesso troppo generiche.
- Registra l'incertezza. Le domande aperte devono restare visibili e avere una fonte o un owner da consultare.

5. Tecniche di Collaborative Modelling

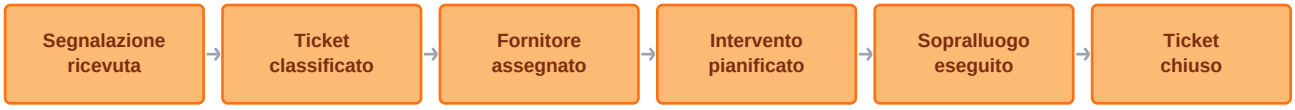
Le tecniche non si escludono: ognuna usa uno zoom differente. La scelta dipende dal tipo di incertezza che stai cercando di ridurre.

Tecnica	Zoom	Problema che risolve	Quando usarla	Quando evitarla / limita
Intervista / grilling	decisioni e ambiguità	sapere disperso, termini vaghi, assenza di	chiarire con uno o pochi esperti	se il problema richiede vedere l'intero
EventStorming	eventi e flusso	processi complessi, dipendenze, flussi end-to-end	sessioni di	CRUD banale o change molto piccol
Domain Storytelling	attori e attività	capire il lavoro reale e le interazioni	workshop con esperti che sanno	se serve subito un dettaglio formale
Example Mapping	regole ed esempi	ambiguità su una singola story/capability	raffinare criteri di accettazione	per mappare un dominio intero

EventStorming è stato creato da Alberto Brandolini come formato di collaborative modelling. Domain Storytelling cattura attività e relazioni tra attori in forma visuale. Example Mapping, introdotto da Matt Wynne, struttura una conversazione usando Story, Rules, Examples e Questions: gli esempi concreti diventano anche una base naturale per i futuri acceptance test.

6. Esempio guidato: manutenzione immobiliare

Immaginiamo una capability: gestire una segnalazione fino alla chiusura dell'intervento. Non partiamo da endpoint o stati enumerati; partiamo da ciò che accade.



- Domanda: chi è autorizzato a chiudere il ticket?
- Regola possibile: un intervento eseguito non è automaticamente accettato.
- Eccezione: cosa succede se il fornitore rifiuta l'incarico o non interviene entro lo SLA?
- Candidate boundary: “intervento” in Pianificazione e “intervento” in Fatturazione rappresentano davvero lo stesso concetto?

Il valore della timeline non sta nei post-it. Sta nelle domande che rende inevitabili.

7. Dal processo alle regole: Domain Storytelling + Example Mapping

Dopo aver visto il flusso, cambia zoom. Chiedi a un esperto di raccontare una storia concreta: “Il condomino segnala un guasto; il gestore verifica la segnalazione; seleziona un fornitore; il fornitore accetta; viene proposta una data; l'intervento viene eseguito; qualcuno verifica l'esito.” La storia fa emergere responsabilità, oggetti scambiati e passaggi reali.

Poi scegli una capability, per esempio “chiudere un ticket”, e fai Example Mapping:

Tipo	Esempio
Story	Chiudere un ticket
Rule	Il ticket è chiudibile solo dopo un esito di intervento valido
Example	Esito OK + nessuna attività pendente -> chiusura consentita
Question	Un esito parziale è sufficiente? Chi può accettarlo?

La domanda rossa è un successo del Discovery: hai scoperto un UNKNOWN prima che diventasse un bug o una decisione arbitraria dell'agente.

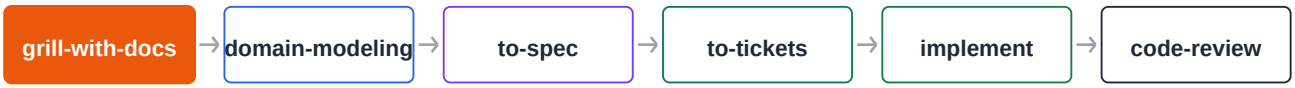
8. Quali artefatti conservare

- CONTEXT.md / glossario: termini canonici, definizioni nel contesto, sinonimi da evitare, parole sovraccariche.
- Mappa del dominio: eventi, attori, capability, candidate boundaries e dipendenze.
- Regole + esempi: invarianti, casi normali, edge case e controesempi concreti.
- Open questions: UNKNOWN espliciti, possibilmente con owner o fonte da consultare.
- ADR selettivi: solo per decisioni difficili da invertire o sorprendenti senza il contesto originale.

Da non produrre troppo presto: schema DB definitivo, endpoint finali, gerarchie di classi o backlog dettagliato. Questi appartengono a fasi successive e devono essere informati dal modello, non sostituirlo.

9. Il workflow di Matt Pocock in questa fase

Il workflow corrente di Matt Pocock colloca grill-with-docs all'inizio del main flow idea -> ship. La skill interroga l'idea, usa il codebase come fonte quando possibile e conserva ciò che impara in CONTEXT.md e negli ADR. La disciplina domain-modeling lavora sotto il flusso per affinare il vocabolario e risolvere termini ambigui.



Strumento	Cosa fa	Problema SE risolto	Principio	Usalo quando	Non usarlo
grill-with-docs	intervista strutturata e stateful, aggiorna CONTEXT.md/ADR	ambiguità e shared understanding + decisioni disperse	Ubiquitous Language / DDD	hai un'idea ma non ancora deciso	hai già chiaro
domain-modeling	disciplina termini e confini	linguaggio incoerente	Ubiquitous Language / DDD	modelli sovraccarichi o ambigui	modello ovvio
to-spec	sintetizza decisioni già prese	perdita di contesto multi-sessione	decision record	decisioni convergenti e stabili	stai ancora esplorando

Il punto metodologico più importante è che to-spec non è una skill di discovery. La documentazione corrente la descrive come una sintesi di decisioni già prese e raccomanda grill-with-docs quando non hai ancora deciso. Per un lavoro piccolo che sta in una singola context window, il flusso può persino saltare la spec e andare direttamente a implement.

10. Cosa fa bene l'AI nel Domain Discovery - e cosa non deve fare

- Bene: generare domande, individuare contraddizioni, proporre controesempi, cercare termini sovraccarichi, raggruppare eventi, sintetizzare conversazioni, mantenere traccia di UNKNOWN e decisioni.
- Bene con cautela: suggerire candidate boundaries, ipotizzare invarianti, confrontare alternative di modello. Devono restare ipotesi finché un esperto o una fonte non le conferma.
- Male: inventare regole di business mancanti e presentarle come fatti.
- Male: sostituire la conversazione con stakeholder reali usando “best practice” generiche.
- Male: trasformare automaticamente ogni sostantivo in entità, ogni evento in microservizio o ogni stato in enum.

Principio operativo: l'AI è un acceleratore epistemico - aiuta a scoprire ciò che non sappiamo - ma l'autorità sulle regole del dominio resta nelle fonti reali: esperti, processi, contratti, dati e sistemi esistenti.

11. Quando fermarsi: proporzionalità del Discovery

Serve più Discovery	Puoi restare leggero
molti “dipende” e casi eccezionali	CRUD semplice e noto
più team, ruoli o sistemi	change piccolo e reversibile
stessa parola con significati diversi	comportamento già fissato da contratto esterno
regole business che cambiano comportamento	problema principalmente tecnico
alto costo di errore o rollback	singolo flusso con pochi edge case

DDD non è un obbligo architetturale. Puoi usare tecniche di Domain Discovery senza adottare microservizi, CQRS, Event Sourcing o un “rich domain model”. DDD strategico è prima di tutto un modo per gestire complessità semantica e confini.

12. Esercizio pratico - 25 minuti

Scegli una capability reale del tuo lavoro oppure una capability inventata. L'obiettivo non è produrre documentazione elegante: è diminuire l'ambiguità.

Tempo	Attività
0-5 min	Scrivi 6-10 eventi di dominio al passato. Vietati nomi di tabelle, endpoint e classi.
5-10 min	Scegli 5 termini ambigui. Definisci cosa significano nel contesto e annota eventuali significati alternativi.
10-15 min	Scegli una capability. Scrivi 3 regole e 1 esempio concreto per ogni regola.
15-20 min	Scrivi almeno 3 domande aperte. Marca ogni affermazione KNOWN / ASSUMED / UNKNOWN / DECIDED.
20-25 min	Valuta il “readiness”: quali parti sono abbastanza comprese da passare ai Requirements? Quali richiedono ancora u

Deliverable minimo dell'esercizio: una pagina con timeline eventi, 5 definizioni di dominio, 3 regole con esempi, 3 domande aperte e una tabella KNOWN/ASSUMED/UNKNOWN/DECIDED.

13. Checklist di uscita dal Domain Discovery

- Possiamo descrivere il processo con il linguaggio del dominio senza ricorrere a termini tecnici?
- Le parole principali hanno significato condiviso o sappiamo esplicitamente dove cambiano significato?
- Abbiamo esempi concreti che mostrano come funzionano le regole?
- Le eccezioni principali sono visibili?
- Le domande aperte sono marcate come UNKNOWN e non nascoste in una frase assertiva?
- Sappiamo chi o cosa può risolvere ogni UNKNOWN importante?
- Abbiamo abbastanza comprensione per formulare requisiti osservabili senza inventare business logic?

Se la risposta all'ultima domanda è sì, il prossimo passo è Requirements Engineering. Se è no, non compensare con più dettaglio tecnico: continua il Discovery.

14. Takeaway

- Il problema viene prima della soluzione. Una specifica precisa del problema sbagliato resta sbagliata.
- Il linguaggio è parte del modello. Ubiquitous Language e Bounded Context sono strumenti operativi per ridurre ambiguità.
- Eventi, storie ed esempi sono sonde. Servono a far emergere regole, eccezioni e confini, non a produrre diagrammi decorativi.
- Le domande aperte sono un output valido. Un UNKNOWN dichiarato è più sicuro di un'assunzione invisibile.
- Con gli agenti, la sequenza conta. Discovery -> decisioni -> spec -> ticket -> implementazione -> verifica.

Collegamento al workflow complessivo: Domain -> Requirements -> Spec -> Test -> Agent -> Verify. In questa lezione abbiamo reso il primo blocco abbastanza solido da poter alimentare il secondo senza perdere significato.

Prossima lezione suggerita: Requirements Engineering - trasformare la conoscenza del dominio in requisiti verificabili. Il focus sarà passare da eventi, regole, esempi e UNKNOWN a requisiti funzionali/non funzionali, invarianti, acceptance criteria e forme controllate come EARS.

15. Fonti autorevoli e letture

Martin Fowler - Ubiquitous Language
https://martinfowler.com/bliki/UbiquitousLanguage.html

Martin Fowler - Bounded Context
https://martinfowler.com/bliki/BoundedContext.html

Martin Fowler - Domain-Driven Design
https://martinfowler.com/bliki/DomainDrivenDesign.html

Alberto Brandolini - EventStorming
https://www.eventstorming.com/

Alberto Brandolini - Introducing EventStorming
https://www.eventstorming.com/book/

Domain Storytelling
https://domainstorytelling.org/

Cucumber - Example Mapping
https://cucumber.io/docs/bdd/example-mapping/

Matt Wynne - Introducing Example Mapping
https://cucumber.io/blog/bdd/example-mapping-introduction/

Matt Pocock / AI Hero - The /to-spec Skill
https://www.aihero.dev/skills-to-spec

Matt Pocock skills flow - ask-matt reference
https://www.ui-skills.com/skills/mattpocock/ask-matt

Nota sulle skill di Matt Pocock: il repository e il workflow sono in rapida evoluzione. Le descrizioni riportate in questa dispensa sono state verificate su documentazione web aggiornata nel 2026; conviene ricontrollarle quando verranno studiate in dettaglio nelle lezioni successive.